

CS⁴¹⁰⁰₅₁₀₀
Shapeshifting Coloring Problems*

Instructions

ALERT: This assignment may carry a high workload, depending on your programming experience. You will need to make several changes to your code for your approach to meet expectations, and succeed on the autograder. **Start early, and read all instructions!**

Academic Integrity

- If you discuss concepts related to this programming assignment with one or more classmates, all parties must declare collaborators in their individual submissions within a code comment. Such discussions must be kept at a conceptual level, and no sharing of actual code is permitted.
- You may use any generative AI tool available to you, as long as it is appropriately cited in a code comment. I recommend using AI tools for debugging or conceptual understanding rather than to produce actual functions.
- Failure to disclose collaborations or use of generative AI will be treated as a violation of academic integrity, and penalties listed in the course syllabus will be enforced.

Reach Out!

If at any point you feel stuck with the assignment, please reach out to the TAs or the instructor, and do so early on! This lets us guide you in the right direction in a timely fashion and will help you make the most of your assignment.

*Developed by Ashwin Bharadwaj, Anio Zhang and Rajagopal Venkatesaramani.
Model AI Assignments, EAAI 2025.

In this assignment, you are given a [PyGame](#) environment (in the [gridgame.py](#) file) that renders a randomly initialized $n \times n$ grid, with some cells pre-filled with one of four colors. Your goal is to build an AI agent that uses **first choice local search** (i.e., picks a random neighboring state at each step) to solve a coloring problem over this grid (see pages 3 and 4 for constraints), such that no two cells that share an edge have the same color, while also minimizing the number of shapes and colors used.

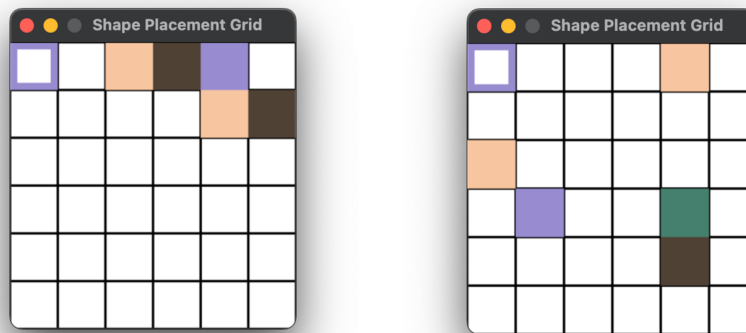
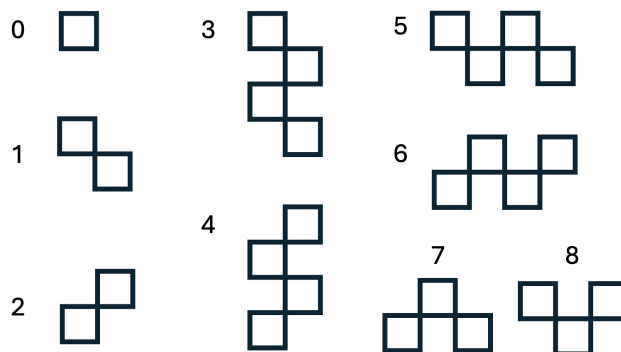


Figure 1: Examples of Initial Configurations

Your agent will attempt to fill the environment by moving a virtual ‘brush’ over this grid and placing colored shapes, where the shape of the brush can be cycled through the following choices (numbered 0-8):



Additionally, each brush can be cycled through one of four colors* (numbered 0-3).



*If, despite our best efforts, the color choices here pose accessibility concerns, they may be edited in the [gridgame.py](#) file. Alternatively, please reach out to us, and we’d be happy to help edit them!

Start by getting a sense of how the environment works by directly running the `gridgame.py` file using Python. You can interact with the game using your keyboard (instructions will be shown in your terminal/IDE console). Spend some time coloring the grid, and note down what behaviors you may want to incentivize when implementing your local search agent's objective function.

Sections 1 and 2 below must be completed and uploaded to Gradescope (ideally with successful autograder execution) before the walkthrough with a TA. The walkthrough for this assignment will consist of a review of your answers to Section 1, a walkthrough of your code, and 1-2 additional clarifying questions from the TA. Changes to your submission after the walkthrough are permitted. Answers to Sections 1 and 3 should be compiled into a single PDF file for upload.

Section 1 - Design

Q1. What do you consider a 'state' in this environment? How would you define a 'neighboring state' for this problem?

Q2. First-choice local search requires that you move to a random neighboring state. How do you (plan to) implement random neighbor selection? Explain in 1-2 sentences.

Q3. Specify your objective function. Are all constraints modeled through the objective function, or are some treated as explicit rules? Explain your choices.

Q4. Draw an end-to-end overview diagram/flowchart for your (proposed) local search approach. If using any extensions to local search (such as random bad moves or restarts), that workflow must be reflected in your flowchart. [May be hand-drawn and scanned into the PDF.]

Section 2 - Implementation

You will now implement your local search agent within the `hw1.py` file. Your agent must interact with the grid environment **only** using the `execute()` function called from within the `hw1.py` file, with the following argument options (passed as strings):

- `export`: returns the current state of the grid, a list of shapes with positions and colors currently placed on the grid, and a Boolean indicating whether the coloring constraints have been satisfied.
- `up/down/left/right`: move the brush in the specified direction by one cell. The brush starts in the top left corner of the grid when the program is executed.
- `place`: place a shape on the grid, i.e. color the cells covered by the brush in the currently selected brush color.
- `switchshape`: cycle to the next brush shape option.
- `switchcolor`: cycle to the next brush color option.
- `undo`: undo the last placed shape.

Running the `execute()` function with any argument returns six items:

Variable	Data	Description
<code>shapePos</code>	Brush Position	Current location of the brush, (X, Y coordinates) <code>list</code> of size 2
<code>currentShapeIndex</code>	Current Shape Index	Index of the currently selected shape, <code>int</code>
<code>currentColorIndex</code>	Current Color Index	Index of the currently selected color, <code>int</code>
<code>grid</code>	Grid State	Updated state of the ($n \times n$ grid), <code>np.array</code> (shape $n \times n$)
<code>placedShapes</code>	Placed Shapes	List of shapes already placed on the grid, <code>list(int)</code>
<code>done</code>	Coloring Constraints Satisfied?	Boolean indicating if coloring constraints are met, <code>bool</code>

Table 1: Summary of Data in the Shapeshifting Coloring Environment

Key Rules

1. All game interactions must strictly occur through the `execute()` command, which modifies the game's internal grid state. You may use a local copy of the grid for intermediate computations, etc.; however, this copy of the grid, including any changes made to it, may not be used or returned as the final solution from your code. **Do not** replace the internal game state or the `grid` global variable using statements of the form `game.grid = xyz` or `grid = xyz`. The ability to implement agents assuming a black-box environment will be crucial once we get to reinforcement learning in a few weeks!
2. You may use any public helper functions defined within `gridgame.py` by calling them in the `hw1.py` file, but you may **not** use any of the private functions (private functions contain an underscore, i.e., `_` at the start of the function name). Additionally, you may access (look up) grid attributes such as `game.grid`, `game.gridSize`, `game.colors` etc., but you may **not** modify them directly by setting/assigning a desired value.
3. Your goal is to build an AI agent that colors this grid, such that no two adjacent cells share the same color. Further, your goal is to achieve this coloring using as few shapes and (new) colors as possible - a larger brush may cover multiple cells, but counts as one shape. Your final submission **must use first choice hill climbing** to implement your agent. Variants such as random restarts, varying step size, and accepting bad moves with decaying probability, are likely to help. Since a local search may not always converge to the 'optimal' coloring, this assignment is graded on correctness, rather than optimality.

To receive full points, your implementation must a) use a randomness-based local search approach, b) complete the board with no constraint violations, c) reasonably attempt to minimize the number of colors and shapes used, d) follow all assignment rules, and e) on average, lead to a valid coloring of 2 test cases within 10 minutes.

NOTE: The autograder only checks the validity of the final solution. The autograder score is not your final grade - points will manually be deducted if any of the rules/expectations outlined above were not followed in your submission.

4. Adjacent cells are defined as cells that share an edge between them (i.e., diagonally neighboring cells may share the same color, since they only share a vertex). If a brush partially or fully overlaps with an area of the grid that is already colored, the `execute` function with the `place` argument will fail, i.e. the cells will not be overwritten.

5. The environment is built as a Python class. You are only allowed to interact with the environment with the `.execute()` method. Do not use any other method from the class to modify any variables in the class. However, you can use the provided public functions (i.e., the `canPlace()`, `getAvailableColors()` and `checkGrid()` functions) to help you write your solution. Feel free to change the arguments for the constructor in `hw1.py` to make the exercise easier for debugging or harder for a personal challenge.

6. Any hardcoded solutions, or attempts to leverage the autograder's design to maximize points scored will result in an automatic zero on the assignment. Failure to use the `execute()` function to interact with the environment, and/or direct manipulation of `gridgame` variables will result in an automatic zero. Implementations must be optimized for good runtime - Gradescope has an autograder timeout of 10 minutes. Any submissions that do not finish executing in 10 minutes will be treated as incomplete, and will be evaluated for partial credit based on correctness.

Section 3 - Analysis/Reflections

Q1. Did you encounter local optima when testing your implementation? If so, when did this happen (structurally, what was going on with the state)? If not, provide an example of a local optimum that could get the agent stuck, where the only way out would be to backtrack to a previous state.

Q2. Is your solution optimal or near-optimal? Explain why/why not.

Academic Integrity: Remember to cite any LLM use, and/or discussions you may have had with classmates using a code comment within the `hw1.py` file, or in your answers PDF. A short (1-2 sentences) description is sufficient.

Submission: Upload your answers PDF, the completed `hw1.py` file, and the provided `gridgame.py` file to Gradescope, and ensure the autograder testcases run as intended with no errors. Please make sure you do not rename the files, or upload a zip/directory containing the files; upload the three individual files directly to Gradescope instead.

Beyond the Assignment - Broader Applications

Graph coloring is a versatile problem with many practical applications. Here are some example problems that are mathematically equivalent to graph coloring.

1. Scheduling

- In [exam scheduling](#), vertices may represent exams and edges connect exams with common students. Colors represent time slots. A proper coloring ensures no student has conflicting exams. Ideas similar to the different brush types in this assignment may be used to optimize room usage, time between exams, etc.
- Graph coloring can be used to schedule classes, ensuring no conflicts in room assignments or student schedules. Faculty room assignments for teaching courses often follow a similar process.
- Graph coloring can optimize [aircraft maintenance schedules](#), minimizing downtime and maximizing efficiency.

2. Resource Allocation

- In radio and mobile networks, graph coloring can be used to [assign specific frequencies to a set of transmitters](#), while the objective function tries to minimize interference. Vertices represent transmitters, edges may represent potential interference as a result of a given frequency allocation, and colors represent frequencies.
- [Compilers use graph coloring to allocate registers efficiently](#). Variables form vertices, edges connect variables that are simultaneously active. Colors represent registers.
- Sudoku puzzles are a resource allocation problem that can be modeled as graph coloring problems. Each cell is a vertex, with edges connecting cells that cannot have the same number. Colors represent the numbers 1-9.

3. Network Analysis

- In various network scenarios, graph coloring can be used to identify and [resolve conflicts](#), such as in wireless sensor networks or traffic management systems.
- Graph coloring may be used in [cybersecurity and network security](#) applications.